



М
Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний інститут імені Ігоря Сікорського»

Навчально-науковий фізико-технічний інститут
Кафедра інформаційної безпеки

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №1

З дисципліни

«КРИПТОГРАФІЯ»

« Експериментальна оцінка ентропії на символ джерела відкритого тексту »

Виконали:

Студенти 3-го курсу, групи ФБ-41

Білогривий М. Р. та Мележик А. А.

Перевірив: Селюх Поліна Валентинівна

Київ 2026

Мета роботи: Засвоєння понять ентропії на символ джерела та його надлишковості, вивчення та порівняння різних моделей джерела відкритого тексту для наближеного визначення ентропії, набуття практичних навичок щодо оцінки ентропії на символ джерела.

Порядок виконання роботи

1. Підготовка тексту

Оберіть довільний текст російською мовою обсягом не менше 1 МБ.

Виконайте попередню обробку тексту відповідно до таких правил:

- усі літери привести до нижнього регістру;
- символи, що не належать до заданого алфавіту, вилучити або замінити пробілами;
- послідовності пробілів та інших розділових символів трактувати як один пробіл;
- для обчислень без пробілів вилучити всі пробіли з попередньо нормалізованого тексту.

Зафіксуйте правила попередньої обробки, використані у вашій програмі.

 **Война и мир. Том 1**

Ми обрали «Война и мир»

```
1  import re
2
3  RUSSIAN_LETTERS = "абвгдежзийклмнопрстуфхцщъыьэюя"
4  ALPHABET_WITH_SPACE = set(RUSSIAN_LETTERS + " ")
5
6
7  def preprocess(raw_text: str, keep_spaces: bool = True) -> str:
8      text = raw_text.lower()
9      text = text.replace("ё", "е")
10
11      text = "".join(ch if ch in ALPHABET_WITH_SPACE else " " for ch in text)
12
13      text = re.sub(r" +", " ", text).strip()
14
15      if not keep_spaces:
16          text = text.replace(" ", "")
17
18      return text
19
20
21  raw = open("voyna_i_mir.txt", encoding="utf-8", errors="ignore").read()
22
23  text_with_spaces = preprocess(raw, keep_spaces=True)
24  text_no_spaces = preprocess(raw, keep_spaces=False)
25
26  print("Довжина сирого тексту:", len(raw))
27  print("Довжина після нормалізації (з пробілами):", len(text_with_spaces))
28  print("Довжина без пробілів:", len(text_no_spaces))
29  print("Перші 300 символів після обробки:")
30  print(text_with_spaces[:300])
```

Зробимо попередню обробку

```

Довжина сирого тексту: 825013
Довжина після нормалізації (з пробілами): 712400
Довжина без пробілів: 597811
Перші 300 символів після обробки:
лев николаевич толстой собрание сочинений в двадцати
Process finished with exit code 0

```

Алфавіт: 32 літери російської мови (без «ё», зведено до «е») + пробіл = 33 символи.

Правила обробки:

- нижній регістр;
- «ё»→«е»;
- всі символи поза алфавітом замінені на пробіл;
- повторні пробіли згорнуті в один;
- окремо створено версію без пробілів.

2. Розрахунок статистичних характеристик тексту

Обрахуємо частоти символів і H_1

```

import math
import re
from collections import Counter

RUSSIAN_LETTERS = "абвгдежзийклмнопрстуфхцчщъыьэюя"
ALPHABET_WITH_SPACE = set(RUSSIAN_LETTERS + " ")

def preprocess(raw_text: str, keep_spaces: bool = True) -> str:
    text = raw_text.lower()
    text = text.replace("ё", "е")
    text = "".join(ch if ch in ALPHABET_WITH_SPACE else " " for ch in text)
    text = re.sub(r" +", " ", text).strip()
    if not keep_spaces:
        text = text.replace(" ", "")
    return text

def char_frequencies(text: str) -> Counter:
    return Counter(text)

def entropy_H1(counts: Counter, total: int) -> float:
    h = 0.0
    for c in counts.values():
        p = c / total
        h -= p * math.log2(p)
    return h

# --- читаємо і обробляємо текст ---
raw = open("voyna_i_mir.txt", encoding="utf-8", errors="ignore").read()
text_with_spaces = preprocess(raw, keep_spaces=True)
text_no_spaces = preprocess(raw, keep_spaces=False)

for label, text in [("з пробілами", text_with_spaces), ("без пробілів", text_no_spaces)]:
    counts = char_frequencies(text)
    total = len(text)
    h1 = entropy_H1(counts, total)

    print(f"\n=== {label} ===")
    print(f"Довжина тексту: {total}")
    print(f"Розмір алфавіту (унікальних символів): {len(counts)}")
    print(f"H1 = {h1:.4f} біт/символ")

    print(f"Топ-5 символів за частотою:")
    for ch, n in counts.most_common(5):
        symbol = " " if ch == " " else ch
        p = n / total
        print(f"  {symbol}: кількість={n}, ймовірність={p:.4f}")

```

Модифікуємо код.

```

=== з пробілами ===
Довжина тексту: 712400
Розмір алфавіту (унікальних символів): 33
H1 = 4.3884 біт/символ
Топ-5 символів за частотою:
_ (пробіл): кількість=114589, ймовірність=0.1608
о: кількість=67581, ймовірність=0.0949
а: кількість=50097, ймовірність=0.0703
е: кількість=47650, ймовірність=0.0669
и: кількість=40173, ймовірність=0.0564

=== без пробілів ===
Довжина тексту: 597811
Розмір алфавіту (унікальних символів): 32
H1 = 4.4712 біт/символ
Топ-5 символів за частотою:
о: кількість=67581, ймовірність=0.1130
а: кількість=50097, ймовірність=0.0838
е: кількість=47650, ймовірність=0.0797
и: кількість=40173, ймовірність=0.0672
н: кількість=39162, ймовірність=0.0655

```

entropy_H1 — реалізує формулу з методички $H_1 = -\sum p_i \cdot \log_2(p_i)$: для кожного символу беремо його ймовірність (частота ÷ довжина тексту), множимо на $\log_2$ від неї, і сумуємо все з мінусом.

Біграми і H₂:

```

import math
import re
from collections import Counter

RUSSIAN_LETTERS = "абвгдежзийклмнопрстуфхцчшщъыьэюя"
ALPHABET_WITH_SPACE = set(RUSSIAN_LETTERS + " ")

def preprocess(raw_text: str, keep_spaces: bool = True) -> str:
    text = raw_text.lower()
    text = text.replace("ё", "е")
    text = "".join(ch if ch in ALPHABET_WITH_SPACE else " " for ch in text)
    text = re.sub(r" +", " ", text).strip()
    if not keep_spaces:
        text = text.replace(" ", "")
    return text

def entropy_from_counts(counts: Counter, total: int) -> float:
    h = 0.0
    for c in counts.values():
        p = c / total
        h -= p * math.log2(p)
    return h

def bigrams_overlapping(text: str):
    # (x1x2), (x2x3), (x3x4)... зміщення на 1 символ
    return [text[i:i + 2] for i in range(len(text) - 1)]

def bigrams_nonoverlapping(text: str):
    # (x1x2), (x3x4)... зміщення на 2 символи
    return [text[i:i + 2] for i in range(0, len(text) - 1, 2)]

```

```
def h2_per_symbol(bigram_list) -> float:
    counts = Counter(bigram_list)
    total = sum(counts.values())
    h_joint = entropy_from_counts(counts, total)
    return h_joint / 2.0 # ділимо на 2, бо H2 має бути "на символ", а не "на біграму"

raw = open("voyna_i_mir.txt", encoding="utf-8", errors="ignore").read()
text_with_spaces = preprocess(raw, keep_spaces=True)
text_no_spaces = preprocess(raw, keep_spaces=False)

for label, text in [("з пробілами", text_with_spaces), ("без пробілів", text_no_spaces)]:
    bg_over = bigrams_overlapping(text)
    bg_nonover = bigrams_nonoverlapping(text)

    h2_over = h2_per_symbol(bg_over)
    h2_nonover = h2_per_symbol(bg_nonover)

    print(f"\n=== {label} ===")
    print(f"Кількість перекривних біграм: {len(bg_over)}")
    print(f"Кількість неперекривних біграм: {len(bg_nonover)}")
    print(f"H2 (перекривні) = {h2_over:.4f} біт/символ")
    print(f"H2 (неперекривні) = {h2_nonover:.4f} біт/символ")

    print(f"Топ-5 біграм (перекривні):")
    for bg, n in Counter(bg_over).most_common(5):
        display = bg.replace(" ", "[ ]")
        print(f" '{display}': {n}")
```

```
=== з пробілами ===
Кількість перекривних біграм: 712399
Кількість неперекривних біграм: 356200
H2 (перекривні) = 3.9858 біт/символ
H2 (неперекривні) = 3.9858 біт/символ
Топ-5 біграм (перекривні):
'o_': 14633
'и_': 12440
'a_': 11710
'e_': 11363
'_п': 10898

=== без пробілів ===
Кількість перекривних біграм: 597810
Кількість неперекривних біграм: 298905
H2 (перекривні) = 4.1569 біт/символ
H2 (неперекривні) = 4.1545 біт/символ
Топ-5 біграм (перекривні):
'то': 9409
'ст': 7763
'на': 7410
'ов': 7238
'ал': 6384
```

Для біграм вираховуємо два варіанти: перекривні біграми та неперекривні.

Варіант тексту H ₂	H ₂ перекривні	H ₂ неперекривні	Різниця
З пробілами	3.9858	3.9858	≈ 0 (0.00004)
Без пробілів	4.1569	4.1545	0.0024

Проаналізуємо отримані результати:

Перекривний спосіб враховує кожну пару сусідніх символів, тому використовує більше даних — 712399 біграм. Неперекривний бере лише кожну другу пару, тому має 356200 біграм. Через це його оцінка H_2 може бути трохи менш стабільною.

Для тексту з пробілами різниця майже нульова ($\approx 0,00004$), оскільки пробіли є частими символами та стабілізують статистику. Без пробілів різниця більша (0,0024), оскільки структура біграм стає менш регулярною.

Який спосіб використовує більше даних?

Перекривний спосіб використовує приблизно вдвічі більше біграм, тому дає точнішу та стабільнішу оцінку H_2 . Неперекривний відкидає половину можливих пар.

3. Дослідження впливу розподілу символів на ентропію

Прогноз: очікуваний порядок зростання H_1 - **Б** < **А** < **В**

Б = 0, бо там завжди один і той самий символ — жодної невизначеності немає.

В — максимальна ентропія, бо всі символи рівноймовірні ($H_1 = \log_2(\text{розмір алфавіту})$).

А — проміжне значення: у природному тексті букви зустрічаються нерівномірно, тому невизначеність менша за максимальну, але більша за нуль.

```
import math
import random
import re
from collections import Counter

RUSSIAN_LETTERS = "абвгдежзийклмнопрстуфхццщщъьэюя"
ALPHABET_WITH_SPACE = set(RUSSIAN_LETTERS + " ")

def preprocess(raw_text: str, keep_spaces: bool = True) -> str:
    text = raw_text.lower()
    text = text.replace("ё", "е")
    text = "".join(ch if ch in ALPHABET_WITH_SPACE else " " for ch in text)
    text = re.sub(r" +", " ", text).strip()
    if not keep_spaces:
        text = text.replace(" ", "")
    return text

def entropy_H1(counts: Counter, total: int) -> float:
    h = 0.0
    for c in counts.values():
        p = c / total
        h -= p * math.log2(p)
    return h

raw = open("voyna_i_mir.txt", encoding="utf-8", errors="ignore").read()
text_no_spaces = preprocess(raw, keep_spaces=False)

LENGTH = 20000
SEED = 42
rng = random.Random(SEED)

# [А] фрагмент природного тексту
seq_A = text_no_spaces[:LENGTH]
```

```
# Б [ повторення одного символу (беремо найчастіший символ джерела)
most_common_char = Counter(text_no_spaces).most_common(1)[0][0]
seq_B = most_common_char * LENGTH

# Б [ випадкова рівномірна послідовність з того самого алфавіту
alphabet = sorted(set(text_no_spaces))
seq_V = "".join(rng.choice(alphabet) for _ in range(LENGTH))

print(f"Алфавіт джерела: {len(alphabet)} символів")
print(f"Найчастіший символ (для Б): '{most_common_char}'")

for name, seq in [("A", seq_A), ("B", seq_B), ("V", seq_V)]:
    counts = Counter(seq)
    h1 = entropy_H1(counts, len(seq))
    print(f"\nПослідовність {name}: довжина={len(seq)}, "
          f"унікальних символів={len(counts)}, H1={h1:.4f} біт/символ")
```

```
"E:\Me\University.KPI\третій курс\Крипта\Lab1\.venv\Scripts\python.exe" "E:\Me
Алфавіт джерела: 32 символів
Найчастіший символ (для Б): 'о'

Послідовність A: довжина=20000, унікальних символів=32, H1=4.4519 біт/символ

Послідовність B: довжина=20000, унікальних символів=1, H1=0.0000 біт/символ

Послідовність V: довжина=20000, унікальних символів=32, H1=4.9989 біт/символ

Process finished with exit code 0
```

Результат: $H_1(A) = 4.4519$, $H_1(B) = 0.0000$, $H_1(V) = 4.9989$ біт/символ.

Наш прогноз повністю підтвердився.

Висновок: нерівномірність розподілу ймовірностей символів прямо зменшує ентропію джерела відносно максимально можливого значення $\log_2(m)$. Чим ближче розподіл до рівномірного, тим вища ентропія; вироджений розподіл (один символ з ймовірністю 1) дає нульову ентропію.

4. Дослідження залежностей між символами

Прогноз

1. H_1 для Γ і Δ - не повинні відрізнятися (частоти символів однакові за побудовою, а H_1 рахується тільки з частот, без урахування порядку).
2. H_2 для Γ і Δ - повинні відрізнятися, бо H_2 враховує зв'язки між сусідніми символами, а порядок символів у Γ і Δ принципово різний.
3. Менше H_2 очікуємо у Δ - періодична структура (типу abababab) робить наступний символ майже повністю передбачуваним за попереднім.
4. Перекривні й неперекривні H_2 у Δ , ймовірно, також трохи відрізнятимуться одне від одного, але обидва будуть значно нижчі, ніж у Γ .

```

import math
import random
import re
from collections import Counter

RUSSIAN_LETTERS = "абвгдежзийклмнопрстуфхцшщъыьэюя"
ALPHABET_WITH_SPACE = set(RUSSIAN_LETTERS + " ")

def preprocess(raw_text: str, keep_spaces: bool = True) -> str:
    text = raw_text.lower()
    text = text.replace("ё", "о")
    text = "".join(ch if ch in ALPHABET_WITH_SPACE else " " for ch in text)
    text = re.sub(r" +", " ", text).strip()
    if not keep_spaces:
        text = text.replace(" ", "")
    return text

def entropy_from_counts(counts: Counter, total: int) -> float:
    h = 0.0
    for c in counts.values():
        p = c / total
        h -= p * math.log2(p)
    return h

def bigrams_overlapping(text):
    return [text[i:i + 2] for i in range(len(text) - 1)]

def bigrams_nonoverlapping(text):
    return [text[i:i + 2] for i in range(0, len(text) - 1, 2)]

def h2_per_symbol(bigram_list):
    counts = Counter(bigram_list)
    total = sum(counts.values())
    return entropy_from_counts(counts, total) / 2.0

raw = open("voyna_i_mir.txt", encoding="utf-8", errors="ignore").read()
text_no_spaces = preprocess(raw, keep_spaces=False)

LENGTH = 20000
SEED = 42
rng = random.Random(SEED)

base = text_no_spaces[:LENGTH]

chars_G = list(base)
rng.shuffle(chars_G)
seq_G = "".join(chars_G)

counts = Counter(base)
symbols_sorted = sorted(counts.keys())
remaining = dict(counts)
pattern = []
while sum(remaining.values()) > 0:
    for s in symbols_sorted:
        if remaining.get(s, 0) > 0:
            pattern.append(s)
            remaining[s] -= 1
seq_D = "".join(pattern)

for name, seq in [("G", seq_G), ("D", seq_D)]:
    c = Counter(seq)
    h1 = entropy_from_counts(c, len(seq))
    h2_over = h2_per_symbol(bigrams_overlapping(seq))
    h2_nonover = h2_per_symbol(bigrams_nonoverlapping(seq))
    print(f"\nПослідовність {name}: довжина={len(seq)}")
    print(f"    H1 = {h1:.4f}")
    print(f"    H2 (перекривні) = {h2_over:.4f}")
    print(f"    H2 (неперекривні) = {h2_nonover:.4f}")

```

Створимо код, який обчислить H_1 та H_2 для обох послідовностей

Послідовність G: довжина=20000

$H_1 = 4.4519$

H_2 (перекривні) = 4.4352

H_2 (неперекривні) = 4.4156

Послідовність D: довжина=20000

$H_1 = 4.4519$

H_2 (перекривні) = 2.7247

H_2 (неперекривні) = 2.6760

Варіант	H_1	H_2 (перекр.)	H_2 (неперекр.)
Г (випадковий порядок)	4.4519	4.4352	4.4156
Д (періодична структура)	4.4519	2.7247	2.6760

Результат:

- $H_1(G) = H_1(D) = 4.4519$ біт/символ — збіглися повністю.
- $H_2(G, \text{перекр.}) = 4.4352$, $H_2(D, \text{перекр.}) = 2.7247$ біт/символ — різниця 1.71 біт/символ.

Пояснення.

Однакові частоти окремих символів у Г і Д дають однаковий H_1 , оскільки ця величина не враховує взаємне розташування символів - тільки те, скільки разів кожен символ трапився. Натомість H_2 враховує залежність між сусідніми символами (умовну структуру пар), а в Г і Д ця структура принципово різна: у Г порядок повністю випадковий (пара символів практично непередбачувана, окрім самих частот), а в Д символи йдуть за жорстким циклічним патерном, тому знання одного символу майже однозначно визначає наступний. Це і показує суть H_2 - вона «бачить» те, чого не бачить H_1 : залежності всередині послідовності, а не лише розподіл окремих символів.

Модель біграм проти моделі незалежних символів: модель незалежних символів (H_1) фіксує тільки «скільки якого символу», а модель біграм (H_2) додатково фіксує «які символи йдуть один за одним», тобто виявляє структурну передбачуваність джерела, недоступну H_1 .

5. Экспериментальна оцінка ентропії джерела

Оцінюємо значення для $H^{(10)}$:

Произвольная часть текста: ателя_обе			Вероятности:	
Использованные буквы:				
Порядок n-граммы:	Введенный символ:	Неравенство для энтропии:	q[1] = 0,56 q[2] = 0,12 q[3] = 0,04 q[4] = 0 q[5] = 0,04 q[6] = 0 q[7] = 0,02 q[8] = 0 q[9] = 0,02 q[10] = 0 q[11] = 0,02 q[12] = 0 q[13] = 0,04 q[14] = 0,02 q[15] = 0 q[16] = 0 q[17] = 0 q[18] = 0,04 q[19] = 0 q[20] = 0 q[21] = 0 q[22] = 0 q[23] = 0 q[24] = 0 q[25] = 0,02 q[26] = 0 q[27] = 0,02 q[28] = 0 q[29] = 0 q[30] = 0,02 q[31] = 0,02 q[32] = 0	
5 символов 10 символов 15 символов 20 символов 25 символов 30 символов 35 символов 40 символов 45 символов 50 символов	Символ по счету:	1,79348354652476 < H < 2,48154193333593		
	Номер эксперимента: 51	Двоичная таблица угаданных символов:		
	Поле ввода символов:	00000000000001000000000000000000 10000000000000000000000000000000 10000000000000000000000000000000 10000000000000000000000000000000 00001000000000000000000000000000		
	Продолжить Другой			
Строка состояния:				

Оцінюємо значення для $H^{(20)}$:

Произвольная часть текста: способны_раздражать_человека_я_бы_не_удивлялся_да_и_кто_я_такой_в_конце_ко			Вероятности:	
Использованные буквы:				
Порядок n-граммы:	Введенный символ:	Неравенство для энтропии:	q[1] = 0,6078431 q[2] = 0,0784313 q[3] = 0 q[4] = 0,0392156 q[5] = 0,0196078 q[6] = 0,0392156 q[7] = 0,0392156 q[8] = 0 q[9] = 0 q[10] = 0,039215 q[11] = 0 q[12] = 0,039215 q[13] = 0 q[14] = 0,019607 q[15] = 0 q[16] = 0,019607 q[17] = 0 q[18] = 0 q[19] = 0,019607 q[20] = 0 q[21] = 0 q[22] = 0 q[23] = 0 q[24] = 0 q[25] = 0,019607 q[26] = 0,019607 q[27] = 0 q[28] = 0 q[29] = 0 q[30] = 0 q[31] = 0 q[32] = 0	
5 символов 10 символов 15 символов 20 символов 25 символов 30 символов 35 символов 40 символов 45 символов 50 символов	Символ по счету:	1,60614252415269 < H < 2,30810993761869		
	Номер эксперимента: 51	Двоичная таблица угаданных символов:		
	Поле ввода символов:	01000000000000000000000000000000 10000000000000000000000000000000 00000010000000000000000000000000 10000000000000000000000000000000 00000000000100000000000000000000		
	Продолжить Другой			
Строка состояния: Вы угадали. Для продолжения опыта нажмите "Продолжить", или "Другой" для выбора другого порядка				

Оцінюємо значення для $H^{(30)}$:

[illegible]

Порівняння отриманих значень:

Оцінка	Отримане значення
H_1	4.3884 (з пробілами)
H_2	3.9858 (перекривні, з пробілами)
$H^{(10)}$	$1.7935 < H < 2.4815$
$H^{(20)}$	$1.6061 < H < 2.3081$
$H^{(30)}$	$1.5504 < H < 2.2626$

Аналіз результатів:

1. Чому наведені оцінки ентропії не повинні обов'язково збігатися;

Оцінки не збігаються, оскільки вони базуються на різних наближених моделях джерела тексту, які враховують різну кількість інформації про контекст. N_1 моделює текст як послідовність повністю незалежних символів, N_2 враховує зв'язок лише з одним попереднім символом, а $N^{(10)}$, $N^{(20)}$ та $N^{(30)}$ враховують пам'ять джерела на глибину 10, 20 та 30 символів відповідно. Оскільки літери в природній мові сильно залежать від свого оточення, моделі з більшою "пам'яттю" дають меншу (точнішу) оцінку невизначеності.

2. Які статистичні залежності враховуються при визначенні N_1 та N_2 .

- а.** При визначенні N_1 враховується виключно одновимірний розподіл ймовірностей — наскільки часто кожна окрема літера зустрічається в тексті в цілому (незалежно від того, де вона стоїть).
- б.** При визначенні N_2 додатково враховуються парні залежності (статистика біграм). Ця оцінка фіксує правила сполучуваності сусідніх символів: наприклад, те, що після приголосної найчастіше йде голосна, після літери "ч" ніколи не стоїть "я", а деякі пари літер взагалі є неможливими в російській мові.

3. Чому врахування довгих залежностей між символами може змінювати оцінку ентропії природної мови;

Природна мова підпорядковується не лише правилам поєднання сусідніх літер, але й глибоким морфологічним, синтаксичним та лексичним правилам, які діють на рівні цілих слів або речень. Контекст із 10–30 символів зазвичай охоплює від одного до кількох слів. Знаючи такий обсяг попереднього тексту, людина або алгоритм може розпізнати корінь слова, граматичний відмінок, префікс або навіть вгадати наступне слово за змістом. Це різко звужує кількість можливих варіантів наступної літери, тим самим радикально зменшуючи ентропію (невизначеність).

4. Яку тенденцію можна спостерігати при переході від H_1 до оцінок, що враховують дедалі довший контекст.

Спостерігається чітка тенденція монотонного спадання оцінки ентропії при збільшенні довжини контексту: $H_1 > H_2 > H^{(10)} > H^{(20)} > H^{(30)}$. За результатами нашого експерименту значення впали з 4.4712 (H_1) до інтервалу 1.55–2.26 ($H^{(30)}$). Тенденція свідчить про те, що чим більше передісторії нам відомо, тим вищою є передбачуваність кожного наступного символу тексту.

6. Оцінювання надлишковості джерела

Надлишковість (R) розраховується за формулою:

$$R = 1 - \frac{H}{H_{\max}}$$

Максимальна ентропія джерела становить $H_{\max} = \log_2(33) \approx 5.0444$ біт/с

Надлишковість:

- $R_1 = 1 - (4.3884/5.0444) = 1 - 0.8699 = 0.1301$ (13.01%)
- $R_2 = 1 - (3.9858/5.0444) = 1 - 0.7901 = 0.2099$ (20.99%)
- $R^{(10)} = 1 - (2.4815/5.0444) = 1 - 0.4919 = 0.5081$ (50.81%)
- $R^{(20)} = 1 - (2.3081/5.0444) = 1 - 0.4575 = 0.5425$ (54.25%)
- $R^{(30)} = 1 - (2.2626/5.0444) = 1 - 0.4485 = 0.5515$ (55.15%)

Якщо враховувати нижні межі ентропії з експериментів, надлишковість для 30 символів сягає майже **69%**.

Порівняння оцінок:

1. Як змінюється оцінка надлишковості при врахуванні залежностей між символами:

Оцінка надлишковості монотонно зростає при збільшенні глибини врахованого контексту. У моделі незалежних символів (H_1) надлишковість становить близько 13%, що відображає лише нерівномірну частоту появи різних літер та пробілу. Врахування парних залежностей (H_2) збільшує її до ~21%, оскільки враховуються базові правила сполучуваності (наприклад, неможливість деяких комбінацій літер). При врахуванні довгих залежностей на рівні 10–30 символів ($H^{(10)} - H^{(30)}$), які охоплюють морфологію, лексику та синтаксис, оцінка надлишковості зростає до 50–55%. Це свідчить про те, що при врахуванні всіх структурних зв'язків тексту більша частина символів стає передбачуваною.

2. Чому природна мова має ненульову надлишковість:

Ненульова надлишковість є фундаментальною властивістю природних мов. Вона виникає через наявність суворих орфографічних, граматичних та синтаксичних правил. Далеко не всі з можливих комбінацій літер є допустимими словами, а не всі комбінації слів утворюють правильні речення. Крім того, надлишковість виконує важливу еволюційну та комунікативну функцію - вона забезпечує завадостійкість. Саме завдяки надлишковості людина здатна читати текст із помилками, відновлювати пропущені літери або розуміти співрозмовника в шумному середовищі.

3. Який зв'язок існує між передбачуваністю наступних символів тексту, ентропією та надлишковістю джерела:

- Передбачуваність і ентропія:

Що вища передбачуваність наступного символу (завдяки знанню попереднього контексту або правил мови), то меншою є невизначеність його появи, а отже, нижчою є ентропія (H).

- Ентропія і надлишковість:

Надлишковість (R) визначається як $1 - H/H_{\max}$. Математично, чим менше значення ентропії H наближається до максимального $H_{\max}$, тим вищим стає значення надлишковості.

Висновок: Висока передбачуваність тексту означає його низьку ентропію, що, у свою чергу, зумовлює високу надлишковість джерела.

7. Аналіз результатів:

1. Чому природний текст має меншу ентропію на символ, ніж випадкова рівномовірна послідовність символів того самого алфавіту?

У природному тексті символи з'являються з різною частотою (наприклад, літера «о» чи пробіл зустрічаються значно частіше за «ф»), що зменшує загальну невизначеність. Випадкова рівномовірна послідовність має максимальну невизначеність, оскільки передбачити наступний символ неможливо. Експеримент 3 (послідовності А та В) підтвердив це: H_1 природного тексту (4.45 біт) виявилось суттєво меншим за H_1 випадкової рівномовірної послідовності (4.99 біт).

2. Яку інформацію про джерело враховує H_2 , але не враховує H_1 ?

H_2 додатково враховує парні статистичні залежності між сусідніми символами (біграми), тобто правила їхньої сполучуваності (наприклад, те, що після приголосної зазвичай йде голосна). H_1 фіксує лише загальну частоту появи окремих ізольованих символів, повністю ігноруючи їхнє взаємне розташування.

3. Чи можуть два тексти мати однакове H_1 , але різне H_2 ? Наведіть приклад із виконаних експериментів.

Так, можуть. У розділі 4 експерименту послідовності Г (випадковий порядок) та Д (періодична структура) були складені з абсолютно однакового набору літер, тому їхні значення H_1 повністю збіглися (4.4519). Проте H_2 у них суттєво відрізнялося (4.4352 для Г проти 2.7247 для Д), оскільки структура пар у періодичній послідовності є жорстко передбачуваною.

4. Як вилучення пробілів вплинуло на отримані значення ентропії? Поясніть отриманий результат.

Вилучення пробілів призвело до збільшення оцінок ентропії (H_1 зросло з 4.3884 до 4.4712, H_2 — з 3.9858 до 4.1569). Пробіл є найчастішим символом, який стабілізує текст і розбиває його на короткі та передбачувані блоки (слова). Його відсутність руйнує звичну структуру, роблячи суцільний потік літер менш упорядкованим та підвищуючи невизначеність.

5. Чому оцінки H_2 , отримані за біграмами, що перетинаються та не перетинаються, можуть дещо відрізнятися?

Перекривний спосіб використовує вибірку, яка дорівнює загальній кількості символів тексту, охоплюючи всі можливі пари сусідів. Неперекривний спосіб (зміщення на два символи) використовує вдвічі меншу вибірку. Ця втрата половини статистичних даних у неперекривному методі призводить до незначних відхилень (наприклад, різниця склала 0.0024 біт для тексту без пробілів).

6. Чому значення H_1 , H_2 , $H^{(10)}$, $H^{(20)}$ та $H^{(30)}$ відрізняються?

Ці оцінки відрізняються, оскільки враховують різну глибину контексту тексту. H_1 розглядає літери ізольовано, H_2 пам'ятає 1 попередній символ, а $H^{(10)}$ - $H^{(30)}$ враховують пам'ять на 10, 20 та 30 літер відповідно. Що довший відомий контекст (який охоплює морфологію та лексику цілих слів), то легше застосувати правила мови для вгадування наступної літери. Це постійно зменшує невизначеність (ентропія падає від ~ 4.38 до ~ 2.26).

7. Який із проведених експериментів найкраще демонструє наявність залежностей між послідовними символами?

Експеримент 4 із порівнянням послідовностей Γ (перемішані символи) та D (періодична структура). Він ізолює вплив структурної залежності, чітко демонструючи, як наявність жорстких правил слідування символів знижує ентропію H_2 при незмінному одновимірному розподілі частот H_1 .

8. Як отримані результати пов'язані з можливістю стиснення природного тексту без втрати інформації?

Значення ентропії визначає теоретичну нижню межу кількості біт, необхідної для кодування одного символу. Оскільки експериментальна ентропія природного тексту з урахуванням глибокого контексту ($H^{(30)}$) падає до ~ 2.26 біт/символ, що значно менше за абсолютний максимум (~ 5.04 біт), текст має високу надлишковість (понад 50%). Це доводить, що за допомогою алгоритмів архівації обсяг природного тексту можна зменшити щонайменше вдвічі без жодної втрати початкової інформації.

Висновки: Виконавши лабораторну роботу, ми експериментально дослідили ентропію джерела відкритого тексту та оцінили його надлишковість. Було встановлено, що природний текст має значно меншу невизначеність, ніж випадкова рівноймовірна послідовність символів,

завдяки нерівномірній частоті появи літер. Ми переконалися, що врахування статистичних залежностей між символами робить текст більш передбачуваним: модель біграм (H_2) додатково знижує ентропію за рахунок базових правил сполучуваності сусідніх літер, а врахування глибокого контексту (10–30 символів), де діють правила лексики та синтаксису, стрімко мінімізує експериментальну ентропію до рівня близько 2.26 біт/символ. Розрахунки довели, що письмовий текст має високу надлишковість (понад 50% при врахуванні довгого контексту), яка забезпечує завадостійкість мови та гарантує можливість стиснення даних без втрати інформації щонайменше вдвічі. Крім того, експерименти наочно продемонстрували важливу роль пробілу як стабілізуючого елемента, вилучення якого підвищує загальну невизначеність тексту, та довели, що наявність жорсткої внутрішньої структури (періодичності) здатна радикально зменшити H_2 навіть при незмінних частотах окремих символів H_1 .